

チーム開発と綺麗なコードの教科書

AIを活用しながら、読みやすく保守しやすいコードを共同執筆するための実践ガイド

第1章：綺麗なコードを書く5つの原則

「動くコード」と「綺麗なコード」の違いは、「**未来の自分や開発パートナーが読んだ瞬間に意味が理解できるか**」です。チーム開発でコードが崩壊しないための5つの基本ルールをマスターしましょう。

原則1：1つの関数（function）には1つの仕事だけさせる

関数が何百行にも及ぶと、どこで何をしているか追えなくなります。「～して、～して、さらに～する」関数は分割のサインです。

```
// ❌ 悪い例：1つの関数で全部処理している
function handleClick() {
  // プレイヤーの位置を更新
  player.x += 10;
  // スコア計算
  score += 100;
  document.getElementById('score').innerText = score;
  // 敵の生成
  let enemy = document.createElement('div');
  // ...あと50行続く
}

// ✅ 良い例：役割ごとに関数を分ける
function handleClick() {
  movePlayer(10);
  addScore(100);
  spawnEnemy();
}
```

原則2：マジックナンバー（謎の数字を直書き）を排除する

コード内に直接`800`や`0.05`などの数字を書くと、後から「何の数字だっけ？」となります。ファイル先頭で意味のある名前をつけて定数化します。

```
// ❌ 悪い例
if (player.x > 800) { ... } // 800って何？画面幅？制限区域？

// ✅ 良い例
const MAX_SCREEN_WIDTH = 800;
if (player.x > MAX_SCREEN_WIDTH) { ... }
```

原則 3：早期リターン（ガード節）でネストを浅くする

`if` の中に `if` が何重にも入る（ネストが深い）コードは読みにくさの原因です。条件に合わない場合は先に関数を抜ける（returnする）ように書きます。

```
// ❌ 悪い例（ネストが深い）
function processGame(isAlive) {
  if (isAlive) {
    if (!isPaused) {
      // メイン処理...
    }
  }
}

// ✅ 良い例（ガード節でスッキリ）
function processGame(isAlive) {
  if (!isAlive || isPaused) return;
  // メイン処理...
}
```

原則 4：HTML / CSS / JavaScript を混ぜない

JavaScriptの中に長大なHTML文字列を直書きしたり、JSからstyleを直接書き換えすぎたりしないようにします。見た目はCSS、構造はHTML、動的処理はJSと役割をきっちり分けましょう。

原則 5：コメントは「何をしているか」ではなく「なぜそうしたか」を書く

コードを見ればわかること（例：`x += 10 // xに10を足す`）はコメント不要です。なぜその特別な数値を使ったのか、なぜその実装方法を選んだのかの理由（Why）を残します。

第2章：わかりやすい変数名・関数名の付け方

変数名や関数名は、コードにおける「看板」です。名前を見るだけで何が入っているか、何をするのかが推測できるように命名します。

1. 基本となる命名ルール

- **camelCase（キャメルケース）を使う**：小文字で始め、単語の繋ぎ目を大文字にします（例：`playerScore`、`isGameActive`）。
- **具体的な英語を使う**：`data`、`item`、`temp`、`a` などの抽象的な名前は避け、具体名にします。

2. 変数・関数の命名パターン表

種類	命名ルール / 接頭辞	✖ 悪い例	○ 良い例
真偽値 (Boolean)	`is`, `has`, `can`, `should` + 状態	flag, check	isGameOver, hasKey
配列 / リスト	複数形 `s` や `List` をつける	enemy, user	enemies, userList
関数 (Function)	`動詞` + `対象 (名詞)`	score(), update()	addScore(), updatePlayerPosition()
イベントハンドラ	`handle` + イベント名	clickBtn()	handleJumpButtonClick()

💡 命名に迷ったときの特効薬

「AIに『～する関数の名前をcamelCaseで5つ提案して』と聞く」のが最も速く確実です！無理に自分で怪しい名前をつける必要はありません。また、難しすぎる英語は使う必要ありません。その場合はローマ字を使いましょう

第3章：共同編集で困らないAI（Gemini）へのプロンプト技術

AI（Gemini Pro等）に「コード書いて」と大雑把に頼むと、1ファイルに全部詰まった保守不能なコードを出力しがちです。AIを「優秀なコーダー」にするためのプロンプトのコツを伝授します。

1. Geminiに最初に読み込ませる「スタイル指定プロンプト」

新しい機能を作る際、チャットの最初に以下の指示を与えておくと、出力されるコードの品質が劇的に上がります。

📋 【AI初期設定用プロンプト】

あなたはチーム開発に熟練したWebプログラマーです。これから作成するコードは以下の設計ルールを厳格に守ってください。

【設計ルール】

- HTML, CSS, JavaScript は完全に別ファイルに分けること。
- JavaScriptは単一責任の原則に従い、1つの関数には1つの役割のみを持たせること。
- マジックナンバーは排除し、ファイル冒頭で const 定数として宣言すること。
- 変数・関数名は省略せず、具体的で分かりやすい英語（camelCase）にすること。
- 条件分岐のネストは避け、ガード節（早期リターン）を使用すること。

2. 共同開発で使える！場面別プロンプト集

① コード美化（リファクタリング）をお願いするとき

以下のコードを共同開発しやすく綺麗なコードに修正してください。

- ・巨大な関数があれば、小さな関数に分割してください。
- ・命名が分かりにくい部分があれば改善してください。
- ・変更点とその理由を簡潔に解説してください。

[ここに自分のコードを貼る]

② プルリクエストを出す前の「セルフチェック」依頼

仲間と共同開発中のコードです。プルリクエストを提出する前にコードレビューをお願いします。

- 可読性が低い部分や、バグになりそうな潜在的リスクはありますか？
- より洗練された書き方があれば提案してください。

チーム開発成功のGolden Rule

AIが生成したコードは**「そのまま貼る前に一度自分で読んで、関数名と変数の役割が言えるか」**を試そう。言えたらPRを出してOK！